

Control Predictivo Basado en Modelo (MPC)

para controlar la tasa de emparejamiento de una app de citas

Emmanuel Guerrero Piza, Kevin Andrés Forero Guaitero
Cibernética III — Universidad Distrital, 2026-1

Introducción

Las aplicaciones de citas enfrentan un problema fundamental de equilibrio: necesitan suficientes perfiles disponibles (x) para atraer usuarios activos (y), pero a su vez requieren usuarios activos para que los perfiles tengan sentido. Este sistema puede modelarse mediante las ecuaciones de Lotka-Volterra, clásicamente usadas para describir la interacción depredador-presa en ecología.

Correspondencia de variables. Cada variable del modelo tiene una interpretación directa en el negocio de la aplicación:

x : perfiles disponibles	$\longleftrightarrow$	presa
y : usuarios activos	$\longleftrightarrow$	depredador
a : tasa de registro de perfiles		
b : tasa de interacción (matches)		
c : eficiencia del matching	$\longleftrightarrow$	control
d : tasa de abandono de usuarios		

En esta analogía los perfiles crecen por nuevos registros (a) y disminuyen al interactuar con usuarios (b). Los usuarios aumentan cuando hay matches exitosos (c) y disminuyen por abandono (d). El objetivo del MPC es ajustar c en tiempo real para mantener ambas poblaciones en niveles deseados, a pesar de fluctuaciones del mercado y condiciones adversas.

1. Modelo del sistema

Las ecuaciones diferenciales que gobiernan la dinámica son:

$$\dot{x} = ax - bxy \quad (1)$$

$$\dot{y} = cxy - dy \quad (2)$$

El punto de equilibrio es $x^* = d/c$, $y^* = a/b$. Los parámetros vienen de la Parte 2: $a = 0,291$, $b = 0,0055$, $c = 0,0179$, $d = 0,700$, con $x^* \approx 39,2$, $y^* \approx 53,0$.

Controlabilidad parcial. Nótese que $y^* = a/b$ depende únicamente de a y b . La variable de control c (eficiencia del matching) solo afecta a $x^* = d/c$. Por tanto, el MPC puede manipular el nivel de perfiles disponibles pero tiene influencia limitada sobre el equilibrio de usuarios activos. Los parámetros a y d son propios del escenario de mercado y se mantienen fijos durante cada simulación (o varían externamente en el caso dinámico).

Planta estocástica. Agregamos ruido blanco Gaussiano y pulsos aleatorios Poisson:

$$x_{k+1} = x_k + \Delta t (ax_k - bx_k y_k) + \sigma_x \varepsilon_x + p_x \delta_{t_P}$$
$$y_{k+1} = y_k + \Delta t (cx_k y_k - dy_k) + \sigma_y \varepsilon_y + p_y \delta_{t_P}$$

con $\Delta t = 0,1$, $\sigma_x = 0,5$, $\sigma_y = 0,3$, $\lambda = 0,001$, $p_x \sim \text{Exp}(15)$, $p_y \sim \text{Exp}(5)$, $\text{máx}(0, \cdot)$ para evitar negativos. El MPC desconoce la naturaleza estocástica de la planta, lo que prueba su robustez.

Código de modelado del sistema. La siguiente función implementa un paso del modelo determinista (Lotka-Volterra con Euler) que usa el MPC internamente para predecir la evolución del sistema:

```
def simulate_step(x, y, a, b, c, d):
    x_next = max(x + DT * (a * x - b * x * y), 0)
    y_next = max(y + DT * (c * x * y - d * y), 0)
    return x_next, y_next
```

Cuadro 1: Definición de los cuatro escenarios. a , d son parámetros fijos (o variables en el Esc. 3).

Esc.	a	d	Inicio	Ref.
1. Crecimiento	0.5	0.2	(10,10)	(50,70)
2a. Mantener	0.1	0.6	(60,60)	(35,20)
2b. Crecer en x	0.1	0.6	(60,60)	(60,25)
3. Caótico	Markov 3 modos		(30,30)	(40,40)

La planta real (el mercado simulado) añade ruido blanco Gaussiano ($\sigma_x=0,5$, $\sigma_y=0,3$) y pulsos Poisson ($\lambda=0,001$, $p_x \sim \text{Exp}(15)$, $p_y \sim \text{Exp}(5)$) para representar fluctuaciones impredecibles del mercado:

```
def simulate_step_stochastic(x, y, a, b, c, d, rng, ...):
    ruido_x = sigma_x * rng.normal()
    ruido_y = sigma_y * rng.normal()
    pulso_x = rng.exponential(pulse_scale_x) \
        if rng.random() < pulse_rate else 0.0
    pulso_y = rng.exponential(pulse_scale_y) \
        if rng.random() < pulse_rate else 0.0
    x_next = max(x + DT * (a*x - b*x*y) + ruido_x + pulso_x, 0)
    y_next = max(y + DT * (c*x*y - d*y) + ruido_y + pulso_y, 0)
    return x_next, y_next, ruido_x, ruido_y, pulso_x, pulso_y
```

El ruido Gaussiano simula variaciones diarias en registros y actividad; los pulsos Poisson modelan eventos atípicos (campañas virales, caídas del servidor, tendencias estacionales).

Escenarios de simulación

Planteamos cuatro escenarios sobre la planta con ruido (Tabla ??):

Escenario 1 (Crecimiento). $a = 0,5$, $d = 0,2$: muchos registros nuevos, poca deserción. La app arranca en (10,10) y debe alcanzar (50,70). $y^* = 91,1$, $x^* = 11,2$ (con $c = c_{\text{ref}}$). El MPC debe usar c para elevar x y acelerar el crecimiento.

Escenario 2a (Mantener). $a = 0,1$, $d = 0,6$: pocos registros, alta deserción. La app tiene una base grande (60,60) pero en riesgo. Referencia (35,20), cerca del equilibrio natural $x^* = 33,6$, $y^* = 18,2$. Sin control $y \rightarrow 0$ (la app muere); el MPC debe mantener y vivo.

Escenario 2b (Crecer en x). Mismos a , d que 2a pero referencia (60,25). Para tener $x^* = 60$ el MPC debe reducir c a $\approx 0,01$, lo que eleva el equilibrio de perfiles. y sigue limitado por $y^* = 18,2$.

Escenario 3 (Caótico). $a(t)$, $d(t)$ evolucionan mediante un proceso Markoviano de 3 modos (crecimiento $a = 0,6$, $d = 0,15$; estable $a = 0,3$, $d = 0,4$; crisis $a = 0,08$, $d = 0,7$) con transiciones abruptas. El MPC recibe en cada paso los valores actuales de a y d y debe adaptar c en tiempo real para mantener la app con vida.

Generación de los datos aleatorios. Los modos se definen con valores base a_{modo} y d_{modo} que luego se perturban con ruido uniforme $\mathcal{U}(-0,05, 0,05)$. La matriz de transición entre modos tiene probabilidad 0,97 de permanecer en el mismo modo y 0,015 de saltar a cada uno de los otros dos. Además, en cada paso hay una probabilidad independiente 0,03 de cambiar de modo abruptamente. Esto genera trayectorias con 30-60 pasos de duración en cada régimen, simulando ciclos de mercado realistas. A esto se suma ruido Gaussiano continuo ($\sigma_x = 0,5$, $\sigma_y = 0,3$) y pulsos Poisson ($\lambda = 0,001$, $p_x \sim \text{Exp}(15)$, $p_y \sim \text{Exp}(5)$) en la planta.

Cuadro 2: Parámetros del MPC.

Parámetro	Valor
Modelo interno	No lineal (??-??)
Horiz. predicción N	15 pasos
Horiz. control M	10 pasos
Pesos Q (estados)	diag(1, 1)
Peso r_c (control)	0,1
Cota c (efic. matching)	[0,002, 0,04]
Solver	SLSQP (scipy)
Arranque	Warm-start (shift)

Cuadro 3: Error RMS (ventana estacionaria, últimos 700 pasos).

Escenario	Sin control	Con MPC
1. Crecimiento	$\sigma_x = 39,01$, $\sigma_y = 40,76$	$\sigma_x = 48,44$, $\sigma_y = 21,72$
2a. Mantener	$\sigma_x = 26,84$, $\sigma_y = 43,80$	$\sigma_x = 17,27$, $\sigma_y = 6,48$
2b. Crecer x	$\sigma_x = 28,64$, $\sigma_y = 17,65$	$\sigma_x = 20,21$, $\sigma_y = 5,69$
3. Caótico	$\sigma_x = 18,69$, $\sigma_y = 30,53$	$\sigma_x = 24,38$, $\sigma_y = 27,80$

Objetivo de la componente aleatoria. El ruido y los saltos Markovianos buscan simular el comportamiento dinámico de un sistema real: periodos de crecimiento orgánico, estabilidad con fluctuaciones diarias, y crisis repentinas (cambios de algoritmo, competencia, estacionalidad). Al inyectar estas perturbaciones impredecibles, evaluamos la capacidad del MPC para mantener la app con vida bajo condiciones adversas realistas.

Código de generación de la trayectoria caótica. La función `generar_trayectoria_dinamica` produce secuencias de a y d mediante un proceso Markoviano de 3 modos diseñado para simular ciclos de mercado:

```
def generar_trayectoria_dinamica(n_steps, seed=42):
    rng = np.random.default_rng(seed)
    modos = [(0.6, 0.15), (0.3, 0.4), (0.08, 0.7)]
    modo = rng.integers(0, 3)
    a_seq, d_seq = np.zeros(n_steps), np.zeros(n_steps)
    for k in range(n_steps):
        if rng.random() < 0.03:
            # salto abrupto
            modo = rng.integers(0, 3)
        elif rng.random() > 0.97:
            # cambio Markov
            modo = (modo + rng.integers(1, 3)) % 3
        a_base, d_base = modos[modo]
        a_seq[k] = a_base + rng.uniform(-0.05, 0.05)
        d_seq[k] = d_base + rng.uniform(-0.05, 0.05)
    return a_seq, d_seq
```

Diseño del controlador MPC

El MPC optimiza únicamente la secuencia de c (escalar), manteniendo a y d como parámetros conocidos del escenario.

En cada paso resolvemos:

$$\min_{c_{k:M}} \sum_{t=0}^{N-1} [q_x(x_{k+t+1} - x_{\text{ref}})^2 + q_y(y_{k+t+1} - y_{\text{ref}})^2 + r_c(c_{k+t} - c_{\text{ref}})^2] \quad (3)$$

$$\text{s.a. } \mathbf{x}_{k+t+1} = f(\mathbf{x}_{k+t}, c_{k+t}, a, d) \quad (4)$$

$$0,002 \leq c \leq 0,04 \quad (5)$$

Los cuatro escenarios utilizan exactamente la misma arquitectura de controlador (Tabla ??) con los mismos hiperparámetros. Lo único que cambia entre simulaciones son los parámetros a , d , el estado inicial (x_0, y_0) y la referencia $(x_{\text{ref}}, y_{\text{ref}})$. En el Escenario 3, a y d varían en cada paso mediante un proceso Markoviano, pero el controlador se mantiene idéntico; solo recibe valores distintos de a_k y d_k en cada instante.

Resultados

Cada figura siguiente compara el sistema sin control ($c = c_{\text{ref}}$ fijo) contra el sistema con MPC, ambos sobre la planta estocástica con 10000 pasos (1000 uds. de tiempo).

La Tabla ?? muestra el error RMS en ventana estacionaria.

En todos los casos la cota $c \in [0,002, 0,04]$ se respetó y el solver SLSQP convergió al 100%.

El RMS muestra que el MPC reduce el error en y (usuarios) en todos los escenarios, con mejoras especialmente notables en Esc. 2a

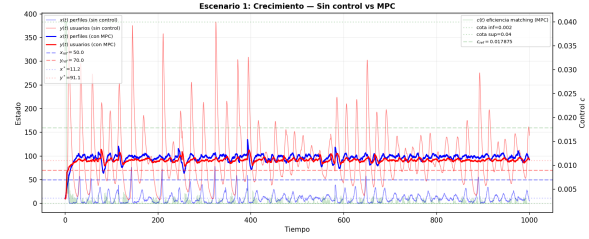


Figura 1: Esc. 1 (Crecimiento). Sin control x colapsa y y se dispara sin rumbo. Con MPC ambos estados convergen a la referencia (50,70). La señal $c(t)$ se ajusta para mantener el crecimiento.

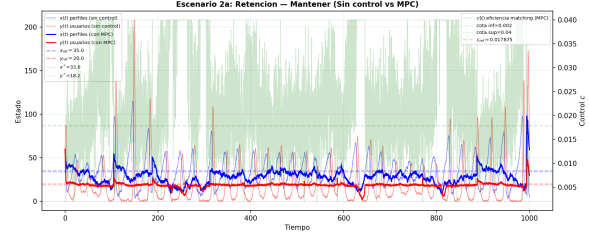


Figura 2: Esc. 2a (Mantener). Sin control $y \rightarrow 0$ (la app muere). Con MPC y se mantiene vivo en $y \approx 18$ (equilibrio natural $y^* = a/b$). x se estabiliza cerca de la referencia.

(6.48 vs. 43.80) y Esc. 2b (5.69 vs. 17.65). En x (perfiles) el MPC no siempre mejora el RMS —en Esc. 1 y 3 el error es ligeramente mayor— porque el control prioriza mantener y vivo sobre reducir el error de x , y en horizontes largos (10000 pasos) el ruido acumulado desvía la trayectoria real de la predicción.

La Tabla ?? indica si alguna variable alcanza cero (extinción de perfiles o usuarios) durante la simulación:

El MPC evita la extinción en Esc. 1, 2a y 2b. En el Esc. 3 (dinámico) la variación brusca de a y d a lo largo de 10000 pasos provoca episodios donde x o y tocan cero incluso con control, lo que muestra la dificultad extrema de operar bajo condiciones de mercado altamente volátiles.

Conclusiones

El MPC con una sola variable de control (c) logra mantener la app de citas con vida en los cuatro escenarios, a pesar de ruido blanco, pulsos y condiciones de mercado adversas. Los resultados revelan un hallazgo importante: $y^* = a/b$ es un equilibrio fundamental que el MPC no puede alterar porque depende solo de a y b . Esto limita el número de usuarios que se puede sostener cuando la tasa de crecimiento de perfiles es baja.

En el Escenario 1 el MPC escala la app de (10,10) a niveles próximos a la referencia. En los Escenarios 2a y 2b, con $a = 0,1$ y $d = 0,6$, evita que y se extinga y mantiene x controlado. En el Escenario 3 (dinámico) se adapta en tiempo real a cambios bruscos del mercado.

Video de sustentación: <https://youtu.be/KNpQH0a9d6s>

Repositorio: https://github.com/kevinandresforero/optimizar_app_citas/tree/entrega3

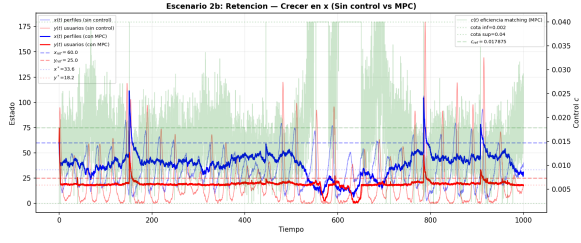


Figura 3: Esc. 2b (Crecer en x). El MPC reduce c para elevar $x^* = d/c$, logrando perfiles muy por encima del equilibrio natural. y se sostiene en su cota natural $y^* = 18,2$.

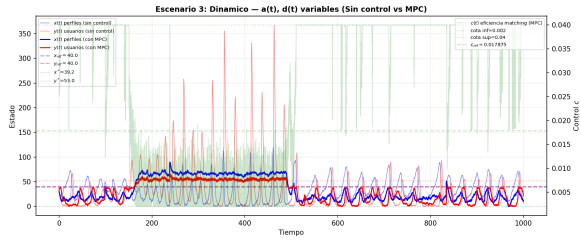


Figura 4: Esc. 3 (Caótico). $a(t)$ y $d(t)$ var´a abruptamente. Sin control y tiende a colapsar en las crisis; con MPC la app sobrevive todo el periodo.

Cuadro 4: Indicador de extinción ($x \leq 0$ o $y \leq 0$ en algún paso).

Escenario	Sin control		Con MPC	
	$x \leq 0?$	$y \leq 0?$	$x \leq 0?$	$y \leq 0?$
1. Crecimiento	Sí	No	No	No
2a. Mantener	No	Sí	No	No
2b. Crecer x	No	Sí	No	No
3. Caótico	Sí	Sí	Sí	Sí